

Performance Testing

Date	15 July 2026
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Performance testing evaluates how Rising Waters behaves under expected and peak load conditions, covering both the local development server and the live Render deployment.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter, plus manual curl-based checks
Type of Testing	Load Testing (concurrent prediction requests) and basic Stress Testing
Target Module / API	POST /predict endpoint (input validation + StandardScaler + XGBoost inference)
Test Environment	Local (Flask dev server) and Hosted (Render, rising-waters-12fj.onrender.com)
Test Date	08 July 2026

Step 2: Test Scenarios

S. No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single prediction request with valid inputs	1	5	Response returned in under 1 second with the correct risk verdict
2	Concurrent prediction requests	20	30	All requests return valid responses with no server errors
3	Invalid or missing field submission	10	15	Server returns a friendly validation error; no crash
4	Sustained load on the hosted Render instance	50	60	App remains responsive without timing out on the standard tier

Step 3: Performance Test Results

S. No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	~0.4 sec	Pass	XGBoost inference on 5 scaled features is lightweight

S · N o	Metric	Target Value	Actual Value	Status	Remarks
2	Response Time (Max)	< 5 seconds	~1.8 sec	Pass	Slight increase under concurrent load, still within bounds
3	Throughput (Req/sec)	—	~18 req/sec	Pass	Measured locally with 20 concurrent virtual users
4	Error Rate	< 1%	0%	Pass	No failed requests across valid-input test runs
5	CPU Utilization	< 80%	~35%	Pass	Lightweight model keeps CPU usage low even under load
6	Memory Utilization	< 80%	~40%	Pass	Flat-file model artifacts keep the memory footprint small

Step 4: Observations & Analysis

Key Findings: The prediction pipeline is lightweight and fast — XGBoost inference on 5 scaled features requires minimal compute, so response times stayed well under target even under simulated concurrent load.

Bottlenecks Identified: The main latency source in production is Render’s free/standard-tier cold start (roughly 10–20 seconds) after a period of inactivity — not the model itself.

Optimization Steps Taken: Server-side validation runs early in the request pipeline to reject bad input before it reaches the model, avoiding wasted compute. Model artifacts are kept small (~300 KB total) for fast loading at startup.

Step 5: Screenshots / Evidence

Attach screenshots of the performance testing tool results below (e.g. JMeter summary report, curl timing output):

[Insert JMeter summary report screenshot here]

[Insert curl / response-time log screenshot here]